NammaPG – User ↔ Role Mapping (Many-to-Many ORM)

1. Problem Statement

A user can have multiple roles and a role can belong to multiple users.

This relationship is classified as Many-to-Many.

2. Database Design Approach

Relational databases require a join table to represent Many-to-Many relationships.

Tables involved:

- users

- roles

- user_roles (join table)

3. Owning Side vs Inverse Side

User entity is the owning side of the relationship.

Role entity is the inverse side.

The owning side defines the join table using @JoinTable.

The inverse side uses mappedBy to avoid duplicate join tables.

4. User Entity Mapping

@ManyToMany defines the relationship.

@JoinTable defines join table name and foreign keys.

FetchType.LAZY ensures roles are loaded only when required.

5. Role Entity Mapping

@ManyToMany(mappedBy = 'roles') tells Hibernate that User owns the relationship.

6. Join Table Structure

user_roles contains:

- user_id (FK → users.id)

- role_id (FK → roles.id)

Composite primary key ensures uniqueness.

7. Why Set is Used

Set prevents duplicate role assignments.

Aligns with relational uniqueness constraints.

8. Lombok & Constructors

@NoArgsConstructor is mandatory for Hibernate.

@AllArgsConstructor supports legacy object creation.

@Builder enables readable and safe object creation.

@Builder.Default preserves default values.

9. ORM Rules Finalized

Each entity maps to one table.

Relationships create join tables.

Only one side owns the relationship.

10. Result

Hibernate manages join table automatically.

Schema is scalable and Spring Security ready.

Next Step: Authentication and Spring Security integration.